

Data Manipulation Language: Simple Queries

Last week you translated an ERD into DDL and stood up your first schema. Now that the tables exist, the next question is how to *ask questions of them*. That is what the **Data Manipulation Language (DML)** is for: a SQL sublanguage for reading and writing rows. Today we focus on reading, using the three clauses that show up in essentially every query you will ever write: **SELECT**, **FROM**, and **WHERE**. We will stick to a single table and build up to combining several conditions.

1. Getting Set Up

You can do today's exercises in either of two ways. The first is to connect to the lab database and load the sample data; the second is to read along on paper and predict the output of each query. Both are fine. If you want to run the queries, follow the steps below.

Connecting

```
psql -h mcsdb.utm.utoronto.ca -U yourutorid -d yourutorid_343
```

If you are off campus, connect to the UofT VPN first (general.vpn.utoronto.ca, group UofT Default). Same password rules as last week.

Loading the schema and sample data

We will query the **Community Library** schema you built in Tutorial 1. If your `practical.ddl` from last week is already loaded, you are good to go. If not, load it first.

```
psql -h mcsdb.utm.utoronto.ca -U yourutorid -d yourutorid_343 -f  
practical.ddl
```

Then download `practical2.dml` from the course site and run it. The file contains **INSERT** statements that put a few `members`, two `branches`, and eight `book_copy` rows into the database so you have something to query.

```
psql -h mcsdb.utm.utoronto.ca -U yourutorid -d yourutorid_343 -f  
practical2.dml
```

If you re-run the data file you may get duplicate-key errors. That is expected and harmless: `psql` will skip the rows that already exist and continue. If you want a clean slate, you can **TRUNCATE** the tables (in the right order!) and reload.

2. What is DML?

DDL (last week) creates the *structure* of the database: tables, types, constraints. **DML** (this week) operates on the *contents* of those tables. The four DML statements you will meet in this course are:

Statement	What it does
SELECT	Read rows from one or more tables (the focus of today).
INSERT	Add new rows to a table.
UPDATE	Modify existing rows.
DELETE	Remove rows.

For the next several tutorials we will live inside **SELECT**. It is by far the richest of the four, and once you can read data fluently, the other three are short.

*A **SELECT** query never changes the database, it only returns a result. You can run the same query a thousand times in a row and your tables will look identical afterwards. This is what makes it safe to experiment with.*

3. The Anatomy of a Simple Query

Every basic SQL query has the same three-clause skeleton. Read it top-to-bottom and it almost sounds like English.

```
SELECT <which columns>
FROM   <which table>
WHERE  <which rows>;
```

- **FROM** names the table you are reading from.
- **WHERE** filters the rows of that table down to the ones you care about.
- **SELECT** picks which columns of those rows to actually return.

The clauses are *written* in the order **SELECT**, **FROM**, **WHERE**, but it is often easier to think about them in the order **FROM** (what am I looking at?) → **WHERE** (which rows survive?) → **SELECT** (what do I want to see?). Writing the query in that mental order will save you many headaches.

Our running example: book_copy

Recall the **book_copy** table from last week's tutorial. After running the sample-data file, it looks roughly like this (a few columns hidden to keep the page tidy):

book_copy_id	title	condition	acquisition_date	branch_name
101	The Bell Jar	worn	2019-04-12	Lakeside
102	A Brief History of Time	good	2021-11-03	Hilltop
103	Beloved	good	2017-06-22	Lakeside
104	Sapiens	new	2023-02-14	Hilltop
105	The Road	damaged	2018-09-30	Lakeside
106	Hyperion	worn	2020-01-08	Hilltop
107	Cosmos	good	2016-05-19	Hilltop
108	Educated	new	2024-08-01	Lakeside

The other columns from your DDL, **isbn**, **replacement_cost_cad**, and **manager_member_id**, are still there, just not shown above. You can run `\d+ book_copy` in `psql` any time to remind yourself of the full schema.

The smallest possible query

```
SELECT * FROM book_copy;
```

The star is shorthand for “every column.” With no **WHERE**, every row is returned. This is the SQL equivalent of “show me the whole table.”

Picking columns

You almost never want every column. Naming them explicitly is clearer and faster:

```
SELECT title, condition
FROM    book_copy;
```

The result has eight rows (one per copy) and two columns. The order of columns in the output follows the order you wrote them in **SELECT**, not the order they appear in the table.

Picking rows

Add a **WHERE** clause to filter:

```
SELECT title
FROM    book_copy
WHERE   branch_name = 'Lakeside';
```

Result: four rows The Bell Jar, Beloved, The Road, Educated.

*SQL string literals use **single quotes**, never double quotes. Double quotes mean something different in PostgreSQL (they quote an identifier, like a column name with spaces). Mixing them up is one of the most common beginner mistakes.*

4. Building Up the WHERE Clause

The **WHERE** clause is just a boolean expression evaluated against each row. The row is kept if the expression is true, dropped otherwise. The expressions are built from the operators below.

Comparison operators

Operator	Meaning	Example
=	equal	condition = 'new'
<>	not equal	condition <> 'new'
<	less than	replacement_cost_cad < 20
>	greater than	replacement_cost_cad > 20
<=	less than or equal	acquisition_date <= '2020-01-01'
>=	greater than or equal	acquisition_date >= '2020-01-01'

These work on numbers, strings, and dates. Dates compare chronologically when written as 'YYYY-MM-DD' strings.

Combining conditions: AND, OR, NOT

Real questions almost always combine several conditions. The three logical connectives behave exactly the way you would expect them to in any programming language.

```
SELECT title
FROM   book_copy
WHERE  acquisition_date >= '2020-01-01'
      AND condition = 'good';
```

This returns books acquired in 2020 or later *and* still in good condition: *A Brief History of Time*.

```
SELECT title, condition
FROM   book_copy
WHERE  condition = 'worn'
      OR condition = 'damaged';
```

This returns the three copies that are not in great shape: *The Bell Jar*, *The Road*, and *Hyperion*.

Precedence: **NOT** binds tightest, then **AND**, then **OR**. So

A OR B AND C means A OR (B AND C).

When in doubt, parenthesise. Nobody loses points for being explicit; people lose points for guessing wrong.

A few more useful operators

Operator	What it does
BETWEEN a AND b	True if the value is in the closed range $[a, b]$. Same as $\geq a$ AND $\leq b$.
IN (v1, v2, ...)	True if the value equals any of the listed values. Cleaner than a chain of ORs.
LIKE 'pattern'	String pattern match. % matches any sequence of characters, _ matches exactly one.
IS NULL	True if the value is missing. <i>Note:</i> you cannot test for null with = NULL.
IS NOT NULL	True if the value is present.

```
SELECT title
FROM   book_copy
WHERE  title LIKE 'The %';    -- starts with "The "
```

Returns *The Bell Jar* and *The Road*.

```
SELECT title
FROM   book_copy
WHERE  replacement_cost_cad BETWEEN 20 AND 30;
```

Returns every copy whose replacement cost is between \$20 and \$30, inclusive.

5. Exercises

Work through these with your neighbour. Predict the result before running the query; then run it and check. All exercises use the `book_copy` table from the previous page.

Warm-up

1. Write a query that returns the `title` and `branch_name` of every copy.
2. Write a query that returns the `title` of every copy whose condition is `new`.
3. Write a query that returns the `title` of every copy acquired *before* 2019.
4. Write a query that returns the `title` and `replacement_cost_cad` of every copy that costs more than \$25 to replace.

Combining conditions

5. Write a query that returns the `title` of every copy owned by the `Lakeside` branch that is in `good` or `new` condition. Write it once using `OR`, then again using `IN`.
6. Write a query that returns the `title` of every copy acquired between 2018 and 2022, inclusive. Try it twice: once with two comparisons joined by `AND`, once with `BETWEEN`.
7. Write a query that returns the `title` of every copy that is *not* in `new` or `good` condition.
8. Write a query that returns the `title` of every copy whose title starts with the word `The`. Hint: `LIKE` with one percent sign.
9. Write a query that returns the `title`, `branch_name`, and `condition` of every copy at `Hilltop` that costs less than \$25 to replace.
10. **Trickier.** Write a query that returns the `title` of every copy that is *either* in `new` condition *or* owned by `Hilltop` and acquired after 2020. Be careful with parentheses — think about what changes if you forget them.

6. Common Pitfalls

A short checklist of the mistakes that come up most often in beginner SQL. If a query is not behaving the way you expect, scan this list first.

1. Using double quotes for strings. Use single quotes: `'fiction'`, not `"fiction"`.
2. Forgetting the semicolon at the end of a statement in `psql`. Nothing runs until `psql` sees one.
3. Writing `= NULL` instead of `IS NULL`. Comparing anything to `NULL` with `=` yields `NULL`, which is not true, so the row is dropped.
4. Mixing up `AND` and `OR`. Read your `WHERE` clause out loud; if it sounds wrong, it probably is.
5. Relying on `AND/OR` precedence without parentheses. When the answer matters, parenthesise.

6. Writing **SELECT *** in real assignments. It is fine while exploring, but in submitted work, list the columns you actually need.

Before You Leave

The priority for today is fluency with the **SELECT/FROM/WHERE** skeleton on a single table. Next week we will extend this to multiple tables and joins, but everything we do later builds on what you practised today. If you finished early, try writing a few of your own questions about the **book** table and trading with a neighbour.

Checklist

1. I can write a query that returns specific columns from a single table.
2. I can filter rows with a **WHERE** clause using **=**, **<>**, **<**, **>**, **<=**, **>=**.
3. I can combine two or more conditions using **AND**, **OR**, and **NOT**.
4. I know which of **AND/OR** binds tighter and how to override it with parentheses.
5. I can use **BETWEEN**, **IN**, and **LIKE** where they are clearer than the long-form alternative.
6. I know how to test for null with **IS NULL** (and not with **= NULL**).
7. I have confirmed I can connect to the lab database, or I am running queries on paper for now.